Análise das Características de Repositórios Populares no GitHub

Autores

Lucas Maia
Leandro Pacheco

Índice

1. Introdução e Hipóteses
2. Questões de Pesquisa (Objetivos)
3. Metodologia
4. Resultados Obtidos
5. Discussão das Hipóteses
6. Tomada de Decisão
7. Conclusão

1. Introdução e Hipóteses

Este trabalho visa analisar os **1.000 repositórios com maior número de estrelas no GitHub**, compreendendo os padrões de desenvolvimento, fluxos de contribuição e ciclos de manutenção presentes nos projetos *open-source* de maior relevância mundial.

O estudo foi desenvolvido ao longo de três sprints. Nas duas primeiras etapas, foram realizadas coletas iniciais e uma primeira consolidação das métricas. Já na **Sprint 3**, foram aplicadas correções metodológicas importantes, refinando o processo de filtragem, coleta e interpretação dos dados. Por isso, este relatório apresenta uma **visão geral consolidada**, incorporando a evolução do experimento e considerando a **Sprint 3 como versão final corrigida da análise**.

A coleta foi feita por meio da **API GraphQL do GitHub**, buscando métricas como:

- idade do repositório;
- total de pull requests aceitas;
- total de releases;
- tempo desde o último push;
- linguagem primária;
- razão de issues fechadas.

Desde o início da análise, observou-se que a amostra dos repositórios mais estrelados é bastante heterogênea. Entre eles estão não apenas projetos de software ativo, mas também repositórios de documentação, listas de recursos, livros, cursos, tutoriais e materiais educacionais. Esses repositórios frequentemente apresentam poucas ou nenhuma PR de código, ausência de releases e baixa atividade real de desenvolvimento, mesmo acumulando milhões de estrelas. Isso cria distorções em métricas tradicionais, razão pela qual adotamos a **mediana como principal medida estatística**.

Antes do início da coleta, elaboramos algumas hipóteses sobre o que esperaríamos encontrar na base de dados:

H1: Projetos massivamente populares tendem a ser antigos, visto que acumular reconhecimento contínuo da comunidade leva vários anos. Esperamos uma idade mediana superior a 5 anos.

H2: A adoção de contribuições externas (Pull Requests) deve ser altíssima, porém assimétrica. Repositórios de frameworks e bibliotecas ativas devem receber um grande volume, enquanto aqueles voltados apenas à documentação e educação devem possuir zero ou poucas contribuições reais de código.

H3: A maior parte dos repositórios lança *releases* frequentemente. Contudo, esperamos forte variação, já que projetos de utilidades puras ou de conteúdo podem não possuir pacotes compilados.

H4: O ecossistema *open-source* do topo do ranking é aquecido. Imaginamos que o tempo desde o último *push* para a grande maioria gire na casa de poucos dias.

H5: Python, TypeScript e JavaScript devem dominar o ranking, por serem linguagens amplamente difundidas e fundamentais em web, dados e desenvolvimento moderno.

H6: A taxa de *issues* resolvidas deve ser elevada, ultrapassando a margem de 80%, dado o zelo dos mantenedores e das comunidades com softwares de amplo alcance.

2. Questões de Pesquisa (Objetivos)

O presente experimento se destina a responder matematicamente e visualmente às seguintes indagações:

RQ 01. Sistemas populares são maduros ou antigos? (Idade do repositório)

RQ 02. Sistemas populares recebem muita contribuição externa? (Total de Pull Requests aceitas)

RQ 03. Sistemas populares lançam *releases* com frequência? (Total de *releases*)

RQ 04. Sistemas populares são atualizados com frequência? (Tempo em dias até a última atualização)

RQ 05. Sistemas populares são escritos nas linguagens mais populares? (Linguagem primária de cada repositório)

RQ 06. Sistemas populares possuem um alto percentual de *issues* fechadas? (Razão entre *issues* resolvidas e registradas)

RQ 07 (Bônus). Sistemas escritos nas linguagens mais populares geram mais PRs, *releases* e recebem atualizações mais recorrentes em relação ao restante da base?

3. Metodologia

Os dados foram minerados automaticamente utilizando um script Python chamado **coleta.py**, responsável por realizar requisições diretas à **API GraphQL do GitHub** com autenticação por token.

Autenticação utilizada:
Authorization: Bearer <token>

O consumo da API foi construído utilizando apenas bibliotecas padrão do Python, sem bibliotecas externas de alto nível. Foram empregados principalmente **urllib.request** e **json**.

3.1 Campos coletados por repositório

Os seguintes campos foram consultados via GraphQL:

nameWithOwner
stargazerCount

```
createdAt
pushedAt
primaryLanguage { name }
pullRequests(states: MERGED) { totalCount }
releases { totalCount }
issues(states: OPEN) { totalCount }
issues(states: CLOSED) { totalCount }
```

3.2 Evolução metodológica ao longo das sprints

Durante as **Sprints 1 e 2**, o experimento já permitiu levantar padrões importantes, mas também revelou limitações metodológicas que precisaram ser corrigidas.

Na **Sprint 1**, por exemplo, a métrica de atualização ainda utilizava o campo **updatedAt**, que é modificado sempre que há novo *star*, *fork* ou *issue*. Isso fazia com que repositórios muito populares parecessem ter sido atualizados no mesmo dia, mesmo sem alteração real de código.

Na **Sprint 2,** essa limitação foi corrigida com a adoção do campo **pushedAt**, que representa o último *push* efetivo no repositório.

Na **Sprint 3**, houve um refinamento ainda mais rigoroso da base, com aplicação de critérios de filtragem para remover repositórios que, embora populares, não representavam software ativo de forma consistente.

3.3 Filtros impostos na Sprint 3

Com o amadurecimento da análise, observou-se que diversos itens entre os mais estrelados do GitHub não eram softwares propriamente ditos, o que distorcia métricas de contribuição e manutenção. Assim, a Sprint 3 passou a excluir do dataset qualquer repositório que:

1. fosse originado de um fork;
2. tivesse menos de 100 KB em disco;
3. estivesse arquivado;
4. não apresentasse linguagem primária definida.

Esses filtros permitiram construir uma amostra mais coerente com o objetivo do experimento: analisar a dinâmica de repositórios populares com atividade real de desenvolvimento.

3.4 Estratégia de paginação e coleta

O cursor da API GraphQL foi utilizado para navegar pelas páginas. Um intervalo de **0,5 segundo entre requisições** foi inserido para respeitar os limites da API.

Além disso, o **PAGE_SIZE foi definido como 10**, evitando erros **502 Bad Gateway** em consultas com múltiplos campos de alta complexidade.

Na Sprint 3, como os filtros passaram a descartar uma quantidade relevante de repositórios da primeira coleta, foi necessário adotar uma estratégia complementar: identificar a quantidade de estrelas do último repositório processado e reiniciar novas consultas com a restrição **stars <= LAST_STAR**, permitindo continuar a coleta até atingir exatamente **1.000 repositórios válidos**.

3.5 Métricas derivadas calculadas

Métrica	Cálculo
age_years	(data_hoje - createdAt).days / 365.25
days_since_update	(data_hoje - pushedAt).days (mínimo 0)
issues_closed_ratio	closed / (open + closed)

3.6 Análise e visualização

O script **gerarHTML.py** lê o CSV gerado e calcula estatísticas descritivas como média, mediana, mínimo e máximo, usando o módulo **statistics** da biblioteca padrão.

Além disso, foi gerado um **dashboard HTML interativo com Chart.js**, permitindo visualizar de forma mais clara a distribuição dos dados.

Devido à presença de assimetria e outliers, a **mediana foi adotada como principal métrica de interpretação**, especialmente para PRs e releases.

3.7 Limitações

Mesmo com os refinamentos da Sprint 3, algumas limitações permanecem:

- a amostra considera apenas os repositórios mais estrelados, não representando o GitHub como um todo;

- repositórios de conteúdo ainda podem aparecer entre os mais populares, ainda que parte deles tenha sido removida pelos filtros;
- o processo de coleta depende das limitações da API, exigindo paginação cuidadosa e intervalos entre requisições.

4. Resultados Obtidos

A análise foi construída progressivamente ao longo de três sprints. A seguir, apresentamos os resultados de cada etapa e destacamos que a **Sprint 3 representa a versão final corrigida do experimento**.

4.1 Sprint 1 — coleta inicial com 100 repositórios

Tabela 1 — Estatísticas gerais (Sprint 1)

Métrica	N	Mediana	Média	Min	Max
Idade (anos)	100	9,18	8,58	0,24	16,48
PRs mergeadas	100	1.703	7.227	0	69.274
Releases	100	15,5	140,52	0	1.000
Dias sem update	100	0	0	0	0
Razão issues fechadas	100	0,91	0,77	0,0	1,0

Observação: na Sprint 1, a métrica de atualização ainda utilizava **updatedAt**, o que fez com que o valor de “dias sem update” aparecesse zerado para todos os repositórios.

Tabela 2 — Top 10 linguagens (Sprint 1)

Linguagem	Repositórios

Python	19
TypeScript	17
N/A	14
JavaScript	11
C++	6
Go	5
HTML	4
Rust	4
Shell	3
Java	3

4.2 Sprint 2 — expansão para 1.000 repositórios

Tabela 3 — Estatísticas gerais (Sprint 2)

Métrica	N	Mediana	Média	Min	Max
Idade (anos)	1.000	8,38	8,19	0,13	17,9
PRs mergeadas	1.000	739	3.955	0	94.643
Releases	1.000	40,5	120,64	0	1.000
Dias sem update	1.000	2	111,99	0	2.284

Razão issues fechadas	1.000	0,87	0,77	0,0	1,0

A Sprint 2 já refletiu uma melhora importante em relação à anterior, especialmente pela substituição de **updatedAt** por **pushedAt**.

Tabela 4 — Top 10 linguagens (Sprint 2)

Linguagem	Repositórios
Python	200
TypeScript	160
JavaScript	115
N/A	95
Go	77
Rust	54
Java	47
C++	46
C	25
Jupyter Notebook	23

Tabela 5 — Medianas por linguagem (Sprint 2)

Linguagem	Repos	Mediana PRs	Mediana Releases	Mediana dias sem update
Python	200	631	23,5	3
TypeScript	160	2.582,5	158	0
JavaScript	115	576	40	5
N/A	95	129	0	129
Go	77	1.690	132	0

4.3 Sprint 3 — versão final corrigida do experimento

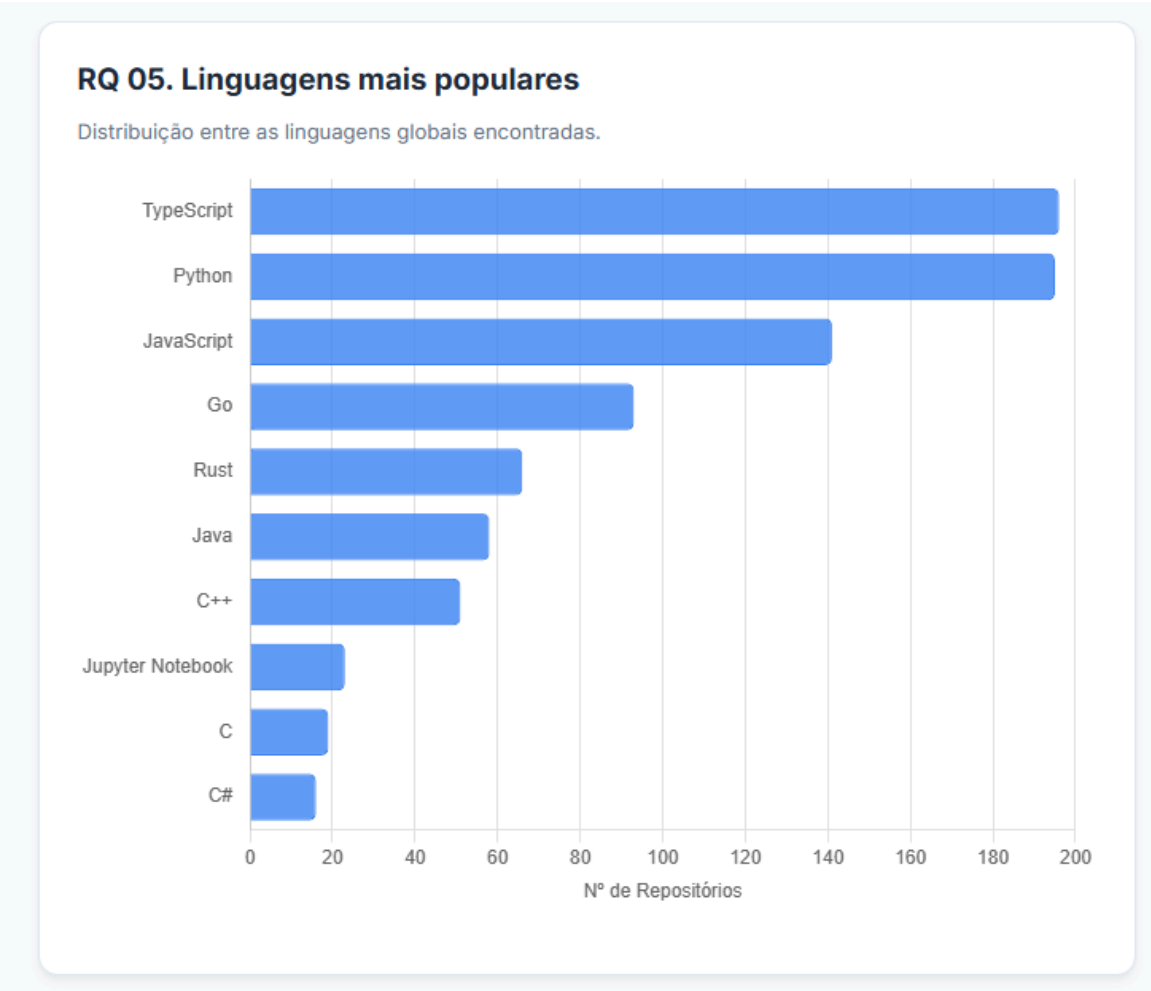
Na Sprint 3, foram aplicadas as correções metodológicas finais, especialmente a filtragem mais rigorosa da base. Isso tornou a amostra mais consistente com o objetivo da pesquisa, reduzindo a influência de repositórios que não representavam efetivamente software ativo.

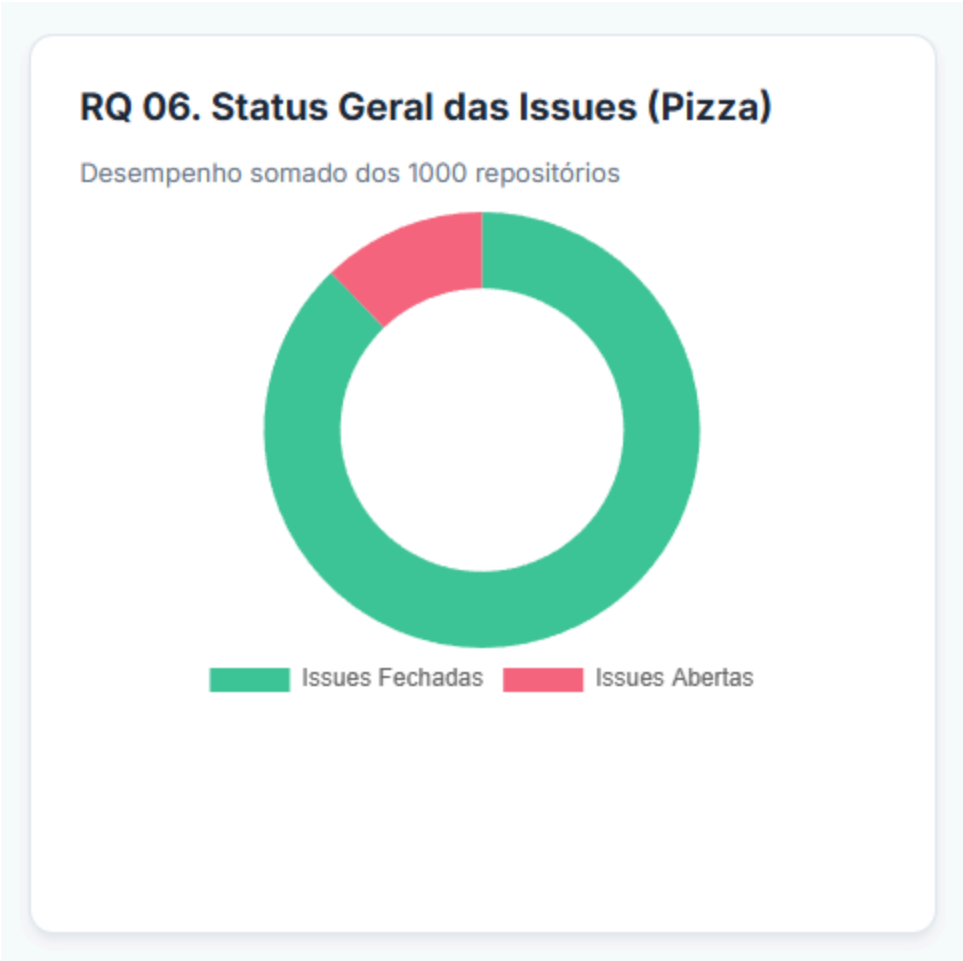
Tabela 6 — Estatísticas gerais (Sprint 3)

Métrica estudada	Respondentes (N)	Mediana	Min	Max
Idade em anos	1.000	8,35	0,14	16,94
Total de Pull Requests aceitas	1.000	884	0	114.397
Lançamentos (Releases)	1.000	50,5	0	1.000
Dias sem atualização (Push)	1.000	1,15	0	2.227
Razão de issues resolvidas	1.000	0,88	0,0	1,00

Tabela 7 — Linguagens dominantes e comportamento por linguagem (Sprint 3)

Linguagem	Repositórios	Mediana PRs	Mediana Releases	Mediana dias sem update
Python	185	860	33	1,76
JavaScript	158	808	51,5	3,52
TypeScript	148	2.594	158	0,50
C++	118	971	38,5	1,39
Go	98	1.854	134	0,49





4.4 Comparação geral entre as sprints

Tabela 8 — Evolução dos resultados entre as sprints

Métrica	Sprint 1	Sprint 2	Sprint 3
Idade mediana (anos)	9,18	8,38	8,35
PRs medianas	1.703	739	884
Releases medianas	15,5	40,5	50,5

Dias sem update	0*	2	1,15
Razão de issues fechadas	0,91	0,87	0,88

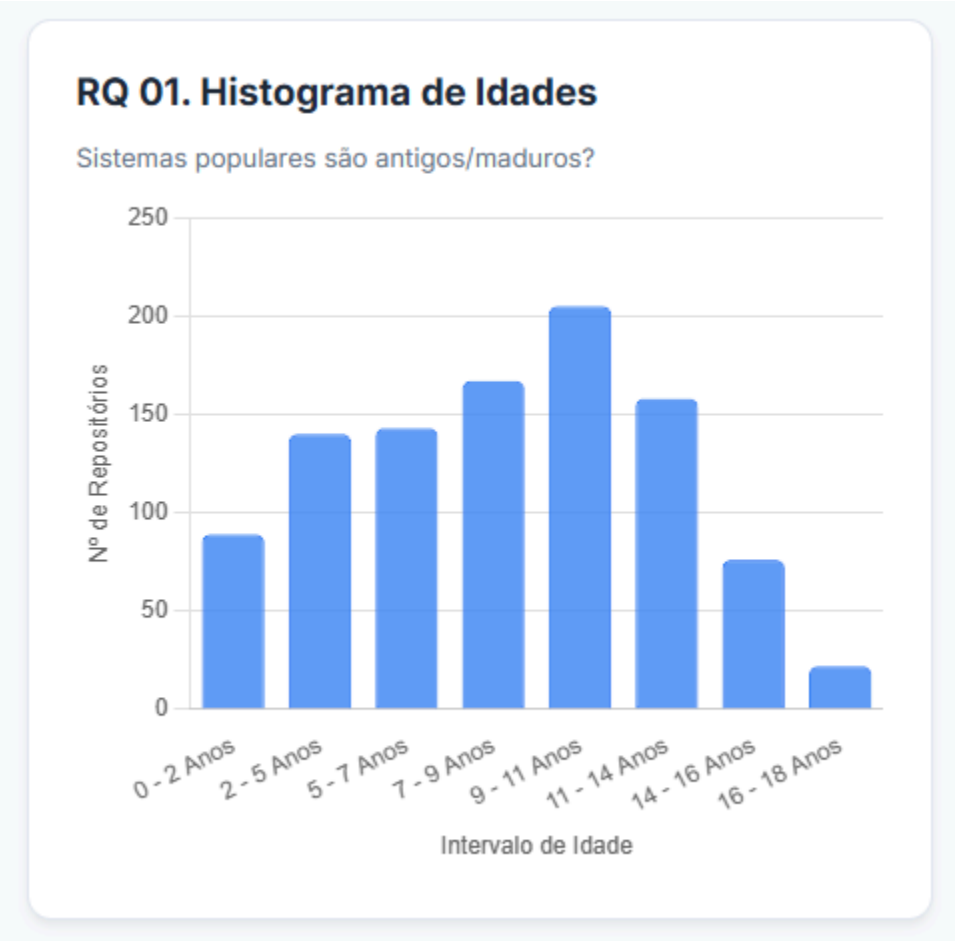
Na Sprint 1, a métrica foi invalidada pelo uso de `updatedAt`.

De forma geral, a Sprint 3 consolidou os resultados do experimento com uma base metodologicamente mais consistente. Houve estabilidade na idade dos repositórios, aumento da mediana de PRs em relação à Sprint 2, crescimento da mediana de releases e manutenção de uma alta taxa de resolução de issues.

5. Discussão das Hipóteses

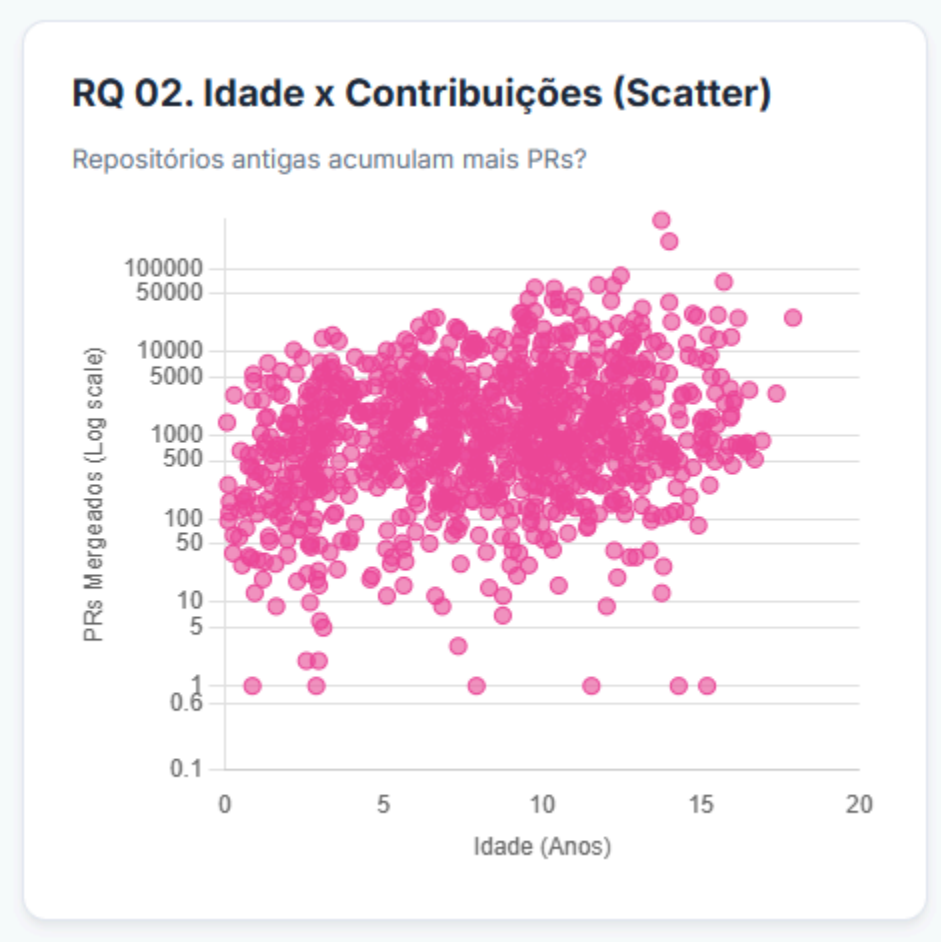
H1 — Repositórios populares são maduros?

Confirmada.
Em todas as etapas, a idade mediana permaneceu próxima de **8 a 9 anos**, indicando que repositórios populares tendem a ser maduros e consolidados.



H2 — Contribuições externas são altas?

Confirmada parcialmente.
A análise final da Sprint 3 mostrou **mediana de 884 PRs**, o que confirma grande participação da comunidade. No entanto, a distribuição é claramente assimétrica, com alguns poucos projetos concentrando um enorme volume de contribuições.



H3 — Repositórios populares lançam releases com frequência?

Confirmada.
Na versão final, a mediana foi de **50,5 releases**, reforçando que projetos populares costumam lançar versões com relativa frequência, embora exista grande disparidade entre tipos de repositório.

H4 — Repositórios populares são frequentemente atualizados?

Confirmada vigorosamente.
Com **mediana de 1,15 dia sem atualização** na Sprint 3, observa-se que os repositórios populares são atualizados quase diariamente, o que evidencia atividade intensa e recorrente.

H5 — Linguagens populares dominam a base?

Confirmada.
Python, JavaScript e TypeScript permaneceram nas posições de maior destaque ao longo das sprints, o que confirma a predominância das linguagens mais populares do ecossistema atual.

H6 — A taxa de issues fechadas é alta?

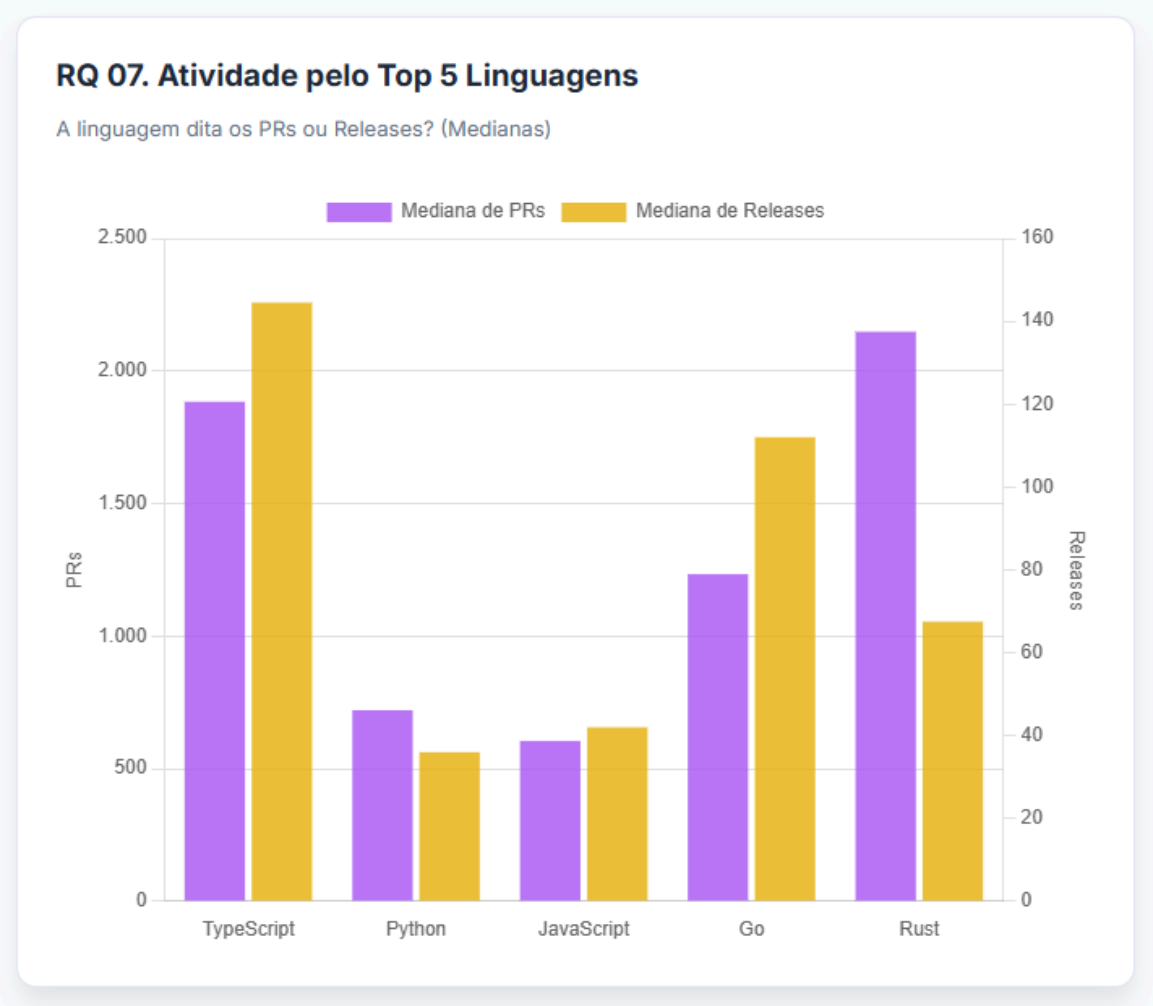
Confirmada.
A mediana de **0,88**, equivalente a **88%**, mostra que a maioria dos projetos populares resolve grande parte das issues registradas, revelando maturidade de manutenção e forte engajamento da comunidade.

6. Tomada de Decisão

Ao longo do desenvolvimento do laboratório, algumas decisões metodológicas se mostraram fundamentais para a qualidade da análise final:

- substituição do campo **updatedAt** por **pushedAt**, para medir atividade real de código;
- adoção da **mediana** como medida principal, devido à presença de forte assimetria;
- redução do **PAGE_SIZE para 10**, evitando erros da API GraphQL;
- inserção de **sleep entre requisições**, respeitando o rate limit;
- aplicação de filtros rigorosos na Sprint 3 para remover repositórios que não representavam software ativo.

A partir dos resultados finais, também foi possível perceber que linguagens como **TypeScript e Go** apresentaram níveis muito altos de PRs e releases, sugerindo ambientes de desenvolvimento mais intensos e pipelines mais ativos de entrega.



7. Conclusão

Este laboratório confirmou que a popularidade no ecossistema moderno de software não é um fenômeno casual, mas sim o resultado de **maturidade, manutenção contínua e participação ativa da comunidade ao longo dos anos**.

A consolidação das três sprints mostrou que os repositórios mais populares do GitHub:

- tendem a ser antigos;
- recebem grande quantidade de contribuições externas;

- são atualizados com alta frequência;
- apresentam elevada taxa de resolução de issues;
- concentram-se majoritariamente em linguagens amplamente difundidas.

Também ficou evidente que métricas simples, como número de estrelas, não são suficientes para representar atividade real de desenvolvimento. Por isso, a Sprint 3 foi essencial para corrigir limitações das etapas anteriores e entregar uma análise final mais robusta.

O uso de filtros metodológicos, da mediana como métrica central e de visualizações interativas permitiu construir um relatório final mais confiável, coerente e academicamente sólido sobre o comportamento dos repositórios *open-source* mais populares do GitHub.